

Project Report: PDF Format Checker

1. Introduction

The **PDF Format Checker** is an automated validation tool designed to evaluate project reports against a departmental standardization template. By leveraging natural language processing and PDF parsing libraries, it checks for various formatting and structural rules to ensure that student or professional reports adhere to strict formatting guidelines before submission.

2. System Architecture & Project Structure

The project is structured into modular components for scalability and maintainability: - **Ingestion (ingestion/)**: Responsible for parsing PDF files utilizing `pdfplumber` and `PyMuPDF`. - **Checks (checks/)**: Contains individual modules for each formatting rule (e.g., layout, fonts, spacing, headings, images, captions, equations, citations, and grammar). - **Reporter (reporter/)**: Generates an Excel-based summary and violation report. - **Utils (utils/)**: Houses format specification constants and error models (Violation dataclass). - **Interfaces**: - `checker.py`: The orchestrator that sequences all checks. - `app.py`: A Streamlit web UI for interactive checking. - `cli.py`: A robust Command-Line Interface for automation.

3. Implemented Checks

The system comprehensively checks the following specifications: - **Layout & Spacing**: A4 page size, Left 1.5" and Right/Top/Bottom 1" margins, 1.5x line spacing, and justified alignment. - **Typography**: Times New Roman font universally. Body font size of 12 pt. - **Heading Hierarchy**: - Level 0 (Chapter Title): 16 pt, UPPERCASE, new page. - Level 1: 14 pt, Bold, UPPERCASE. - Level 2 & 3: 12 pt with specific casing and boldness. - **Figures & Tables**: Checks for ≥ 600 DPI for images, proper axis labels (via OCR), and correct caption placements (below for figures, above for tables). - **Content Rules**: Checks for all 8 mandatory chapters in order. - **Citations & Grammar**: Validates APA 7th format and performs grammar checks using `LanguageTool` with a specialized Computer Science dictionary.

4. Technologies Used

- **PDF Processing**: `pdfplumber`, `PyMuPDF`, `pypdf`, `pdfminer.six`
- **OCR & Image Processing**: `pytesseract`, `Pillow`
- **Natural Language & Grammar**: `language-tool-python`, `regex`
- **User Interfaces**: `streamlit` for the Web UI, `colorama` for CLI output formatting.
- **Reporting**: `openpyxl` for exporting violation data into Excel.

5. Usage & Deployment

The system can be deployed and used in two primary ways: 1. **Command Line Interface (CLI)**: Ideal for batch processing and CI/CD pipelines. It supports custom output paths and skipping specific heavy checks like grammar. 2. **Web UI**: Built with `Streamlit`, it provides a user-friendly drag-and-drop interface running locally on port 8501.

6. Conclusion

The PDF Format Checker provides an efficient, automated solution to a tedious manual validation process. Its modular design allows easy extension with new checks, making it highly adaptable for various formatting templates.